

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

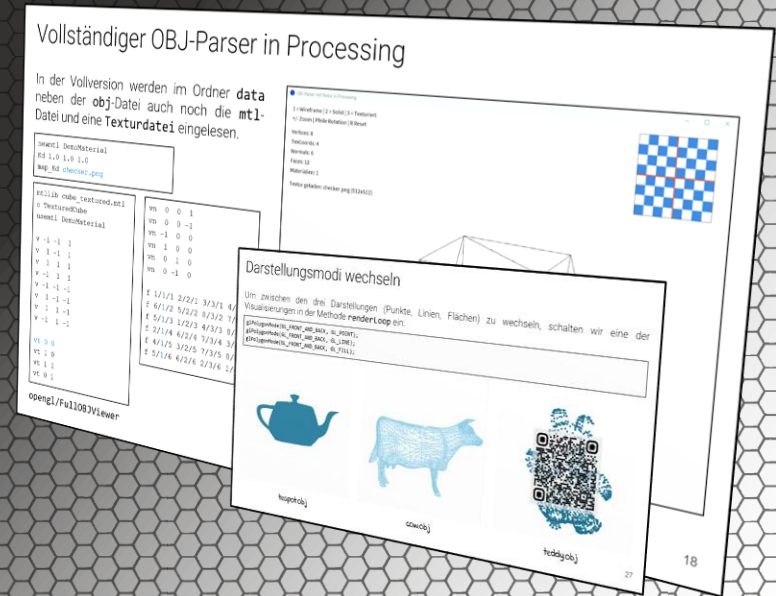
Sommersemester 2026



3D-Modelle: Formate, Parsing und Visualisierung

„Nichts ist so beständig wie der Wandel.“ (Heraklit von Ephesus)

PD Prof. Dr. Marco Block-Berlitz



Ziele und Inhalt

- 3D-Modelle integrieren als zentrales Problem
- Grundidee eines 3D-Modells
- Dreiecke unter der Lupe
- Dreiecke als Basis für 3D-Modelle
- Überblick zu 3D-Formaten
- Struktur des Wavefront-OBJ-Formats
- Faces als Dreiecke
- Minimal-Parser in Processing
- Erweiterung des Parsers um Face-Variationen und Attribute
- Erweiterung des Parsers um Materialien und Texturen
- Vollständiger OBJ-Parser in Processing
- OBJ-Parser: Klassen Face, Triangle und Model
- OBJ-Parser: POGI-Methode loadModel
- Beispielobjekte in der Programmsammlung
- Objekt laden und drehen
- Darstellungsmodus wechseln
- 3D-Modelle bei SketchFab
- Vollständiger OBJ-Parser in Aktion
- 3D-Modelle mit Meshy.AI erzeugen
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben





Es gibt viele **Werkzeuge**,
3D-Modelle komfortabel zu erstellen

3D-Modelle integrieren als zentrales Problem

Es gibt eine Reihe von Werkzeugen, mit denen sich 3D-Objekte komfortabel und schnell erstellen lassen. Maya [1], Cinema 4D [2] und 3DS Max [3] sind dabei sehr verbreitete kommerzielle Produkte, die gerade im Computerspiele- oder Animationsbereich ihre Verwendung finden. Blender bietet hier eine sehr gute kostenlose Alternative [4]. Ein zentrales Problem der Computergrafik besteht darin, 3D-Modelle aus externen Tools in eigene Anwendungen zu integrieren.

Typische Pipeline

- Modellierung (z. B. Blender [4])
- Export in ein Dateiformat (z. B. OBJ)
- Einlesen (Parsing)
- interne Datenstruktur
- Rendering



3ds Max



Maya



CINEMA 4D

Herausforderung: Unterschiedliche Formate und unterschiedliche Komplexität

[1] <https://www.autodesk.de/products/maya/overview>

[2] <https://www.maxon.net/en-us/products/cinema-4d/overview/>

[3] <https://www.autodesk.de/products/3ds-max/overview>

[4] <https://www.blender.org/>

Grundidee eines 3D-Modells

Ein 3D-Modell beschreibt die Form und Eigenschaften eines Objekts im dreidimensionalen Raum [1].

Geometrie

3D-Positionen von Punkten im Raum (Vertices)

Topologie

Verbindung der Punkte zu Kanten und Flächen (Primitive)

Attribute

- Normalen (Beleuchtung)
- Texturkoordinaten
- Materialparameter
- Farben



Wir erinnern uns an eine der letzten Vorlesungen

Dreiecke unter der Lupe

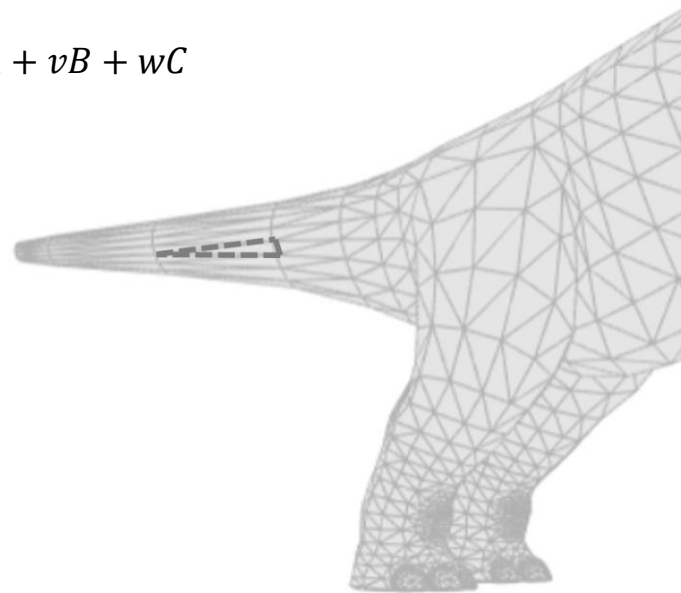
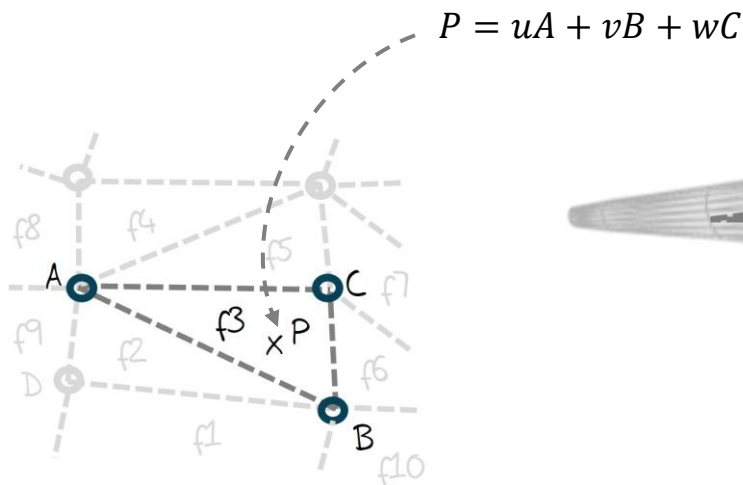
Objekte werden in der **Regel durch Dreiecke** repräsentiert. Eine triangulierte Oberfläche, die aus vernetzten Dreiecken besteht, kann an den Knoten neben den relativen **x**-, **y**- und **z**-Koordinaten auch Informationen zu Farben oder anliegenden Texturen speichern.

Vertex-Liste

```
...  
A 0.0033 0.0057 0.1845 f2 f3 f5 f4 f8 f9  
B 0.0051 0.0058 0.1920 f2 f1 f10 f6 f3  
C 0.0053 0.0045 0.2010 f3 f6 f7 f5  
...
```

Face-Liste

```
...  
f2 A D B  
f3 A B C  
...
```

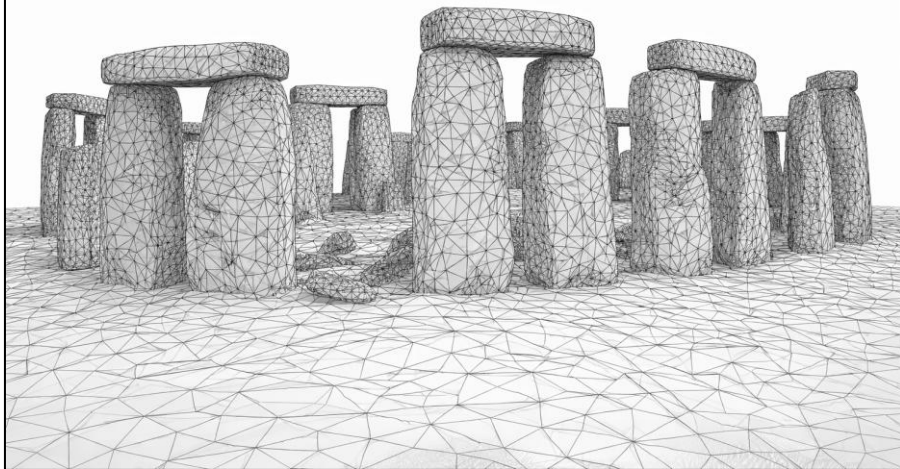


OBJ-Format

Dreiecke als Basis für 3D-Modelle

Polygonnetze werden in der Praxis fast immer in Dreiecke zerlegt [1]. Dreiecke sind immer planar, eindeutig definiert, gut zu interpolieren (Baryzentrische Interpolation) und effizient auf GPUs umzusetzen (alle Modelle werden intern trianguliert). Die Triangulierung komplexer Flächen ist daher immer ein wichtiges Thema.

Mesh bestehend aus
Punkten und Dreiecken



texturiertes Mesh
beleuchtet und mit
Hintergrund



[1] Akenine-Möller T. et al.: „Real-Time Rendering“, CRC Press, 2018

Überblick zu 3D-Formaten

Zur Speicherung von 3D-Modellen existieren zahlreiche Formate mit unterschiedlichen Eigenschaften. Es gibt eine große Anzahl konkurrierender Standards, wobei sich das OBJ-Format, neben PLY und STL, mittlerweile als weit verbreitetes Austauschformat etabliert.

OBJ

textbasiert, einfach zu parsen, keine Kompression, langsames Parsing großer Modelle, getrennte Listen für Positionen, Normalen und Texturen, gut für die Lehre

Archäologische Objektdatenbank mit knapp 5.000.000 Einträgen

PLY

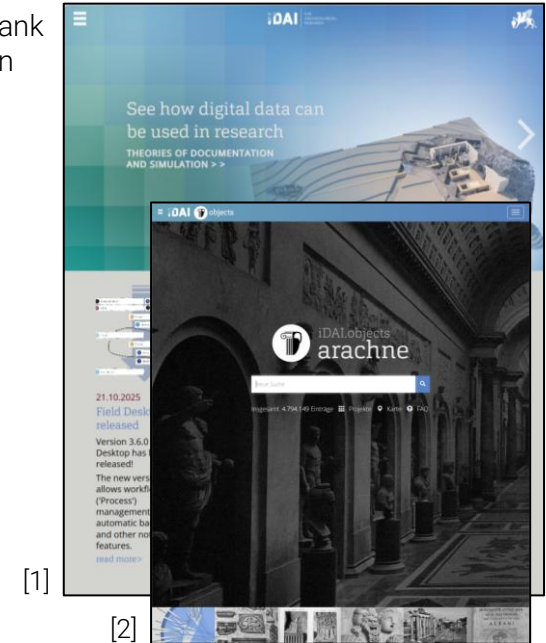
flexibel

STL

nur Geometrie, keine Texturen, geeignet für 3D-Druck

FBX

komplex, umfangreich, Animationen, binär



[1] <https://idai.world>

[2] <https://arachne.dainst.org>

Struktur des Wavefront-OBJ-Formats

Eine OBJ-Datei enthält Listen im Textformat für 3D-Positionen, Texturkoordinaten, Normalen und Vertex-Indizes [1]. Wir geben **3D-Positionen** mit einem **v** für Vertex gefolgt von drei Float-Werten für **x**, **y** und **z** an:

```
v 1.0 -1.5 2.2
```

Texturkoordinaten werden mit **vt** codiert und erhalten zwei Float-Werte für **u** und **v** (es können auch drei Werte für **u**, **v** und **w** angegeben werden, wobei der letzte Parameter optional ist und standardmäßig auf 0 steht):

```
vt 0.3 0.68
```

Wenn wir eine **Normale** angeben wollen, beginnen wir mit **vn** gefolgt von drei Float-Werten für **x**, **y** und **z**:

```
vn 0.0 1.0 0.0
```

Nachdem die Punkte in einer bestimmten Reihenfolge definiert wurden, können wir die Positionen der Punkte in den Listen nutzen, um daraus Flächen aus Dreiecken oder Vierecken zu beschreiben.

Hinweis: **Indizes beginnen bei 1** und nicht bei 0, was eine beliebte Fehlerquelle darstellen kann.

[1] http://www.cs.utah.edu/~boulos/cs3505/obj_spec.pdf

Faces als Dreiecke

Objekte werden meistens über Dreiecke beschrieben. Wenn wir ein „Face“ als Dreieck angeben, dann starten wir mit einem **f** gefolgt von drei Vertex-Beschreibungen, die jeweils drei Indizes **v/vt/vn v/vt/vn v/vt/vn** haben. Dabei bezieht sich der jeweils erste Index auf die [Liste der Positionen](#), der zweite auf die [Liste der Texturkoordinaten](#) und der dritte auf die [Liste der Normalen](#):

```
f 1/1/1 2/13/2 3/2/3
```

Schauen wir uns ein Beispiel an:

```
f 1/2/3
```

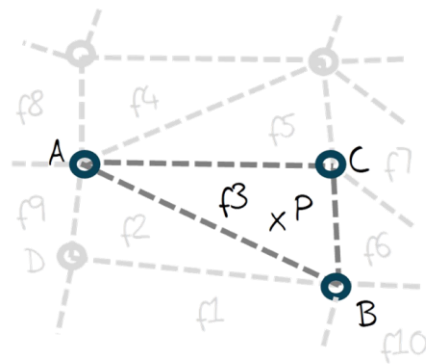
Listen sind getrennt und Indizes unabhängig. Daher nutzt hier Vertex 1 im Face [Position](#) 1, [Texturkoordinate](#) 2 und [Normale](#) 3.

Vertex-Liste

```
...  
A 0.0033 0.0057 0.1845 f2 f3 f5 f4 f8 f9  
B 0.0051 0.0058 0.1920 f2 f1 f10 f6 f3  
C 0.0053 0.0045 0.2010 f3 f6 f7 f5  
...
```

Face-Liste

```
...  
f2 A D B  
f3 A B C  
...
```



OBJ-Format



Schauen wir uns einen Minimal-OBJ-Parser
in Processing an

Minimal-Parser in Processing

Hier sehen wir ein einfaches Beispiel für einen minimalen OBJ-Parser in Processing, der nur **v** und **f** unterstützt und Dreiecke erlaubt.

```
class OBJModel {  
    ArrayList<PVector> vertices;  
    ArrayList<Face> faces;  
  
    ...  
}
```

Processing

```
class Face {  
    int a, b, c;  
    Face(int a, int b, int c) {  
        this.a = a;  
        this.b = b;  
        this.c = c;  
    }  
}
```

```
# Einfacher Würfel aus Dreiecken  
  
v -1 -1 -1  
v 1 -1 -1  
v 1 1 -1  
v -1 1 -1  
v -1 -1 1  
v 1 -1 1  
v 1 1 1  
v -1 1 1  
  
# Vorderseite  
f 5 6 7  
f 5 7 8  
  
# Rückseite  
f 1 3 2  
f 1 4 3
```

```
# Linke Seite  
f 1 5 8  
f 1 8 4  
  
# Rechte Seite  
f 2 3 7  
f 2 7 6  
  
# Oberseite  
f 4 8 7  
f 4 7 3  
  
# Unterseite  
f 1 2 6  
f 1 6 5
```

model.obj

Minimalparser in Processing

In der Minimalversion wird auch nur die in **data** liegende Datei **model.obj** eingelesen.

```
# Einfacher Würfel aus Dreiecken

v -1 -1 -1
v  1 -1 -1
v  1  1 -1
v -1  1 -1
v -1 -1  1
v  1 -1  1
v  1  1  1
v -1  1  1

# Vorderseite
f 5 6 7
f 5 7 8

# Rückseite
f 1 3 2
f 1 4 3
```

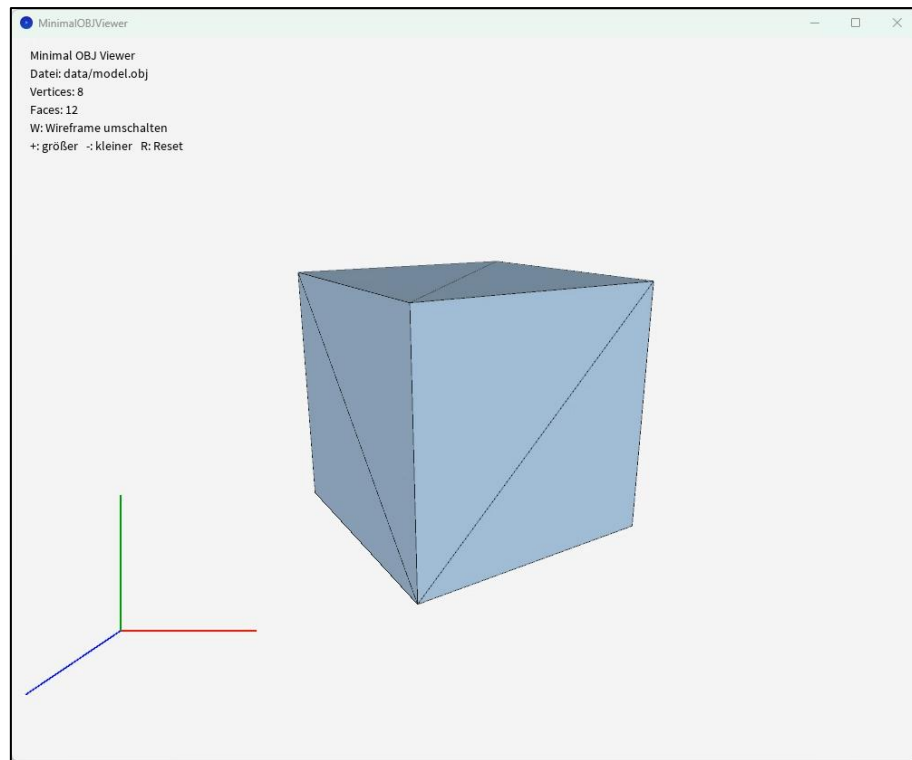
```
# Linke Seite
f 1 5 8
f 1 8 4

# Rechte Seite
f 2 3 7
f 2 7 6

# Oberseite
f 4 8 7
f 4 7 3

# Unterseite
f 1 2 6
f 1 6 5
```

model.obj



Erweiterung des Parsers um Face-Variationen und Attribute

Die größte Herausforderung beim Parsing (neben fehlenden Werten, ungültigen Indizes, ...) liegt in der Interpretation der [Face-Definitionen](#). Schauen wir uns dafür mal ein paar Varianten an [1]:

```
f v v v  
f v/vt v/vt v/vt  
f v//vn v//vn v//vn  
f v/vt/vn v/vt/vn v/vt/vn
```

```
f 1 2 3 4
```

⇒ wird zu Dreiecken (1, 2, 3) und (1, 3, 4). [GPU unterstützt nur Dreiecke](#).

Wir haben es oft mit [unvollständigen Modellen](#) zu tun, es können also Inhalte fehlen und wir müssen mit Standardwerten /-routinen darauf antworten:

```
f 1//3      // keine Textur  
f 1/2       // keine Normale
```

Für komplexe Modelle müssen [Geometrie und Attribute getrennt](#) gespeichert werden (Effizienz, Flexibilität):

```
class VertexRef {  
    int v, vt, vn;  
}
```

Processing

Erweiterung des Parsers um Materialien und Texturen

[Materialdateien](#) erweitern OBJ-Modelle um visuelle Eigenschaften und werden als sogenannte **mtl**-Dateien gespeichert:

- diffuse Farbe (K_d)
- spekulare Reflexionseigenschaften (K_s)
- Texturen (map_ K_d)

```
mtllib material.mtl  
usemtl Material01
```

Nach dem vollständigen Parsing können Modelle mit Texturen dargestellt werden. Texturkoordinaten verbinden 3D-Geometrie mit 2D-Bilddaten über baryzentrische Interpolation:

```
texture(img);  
vertex(x, y, z, u, v);
```

Processing

Ein robuster Parser muss auch ungültige Indizes, fehlende Daten, falsche Formate und nicht triangulierte Faces verarbeiten können.

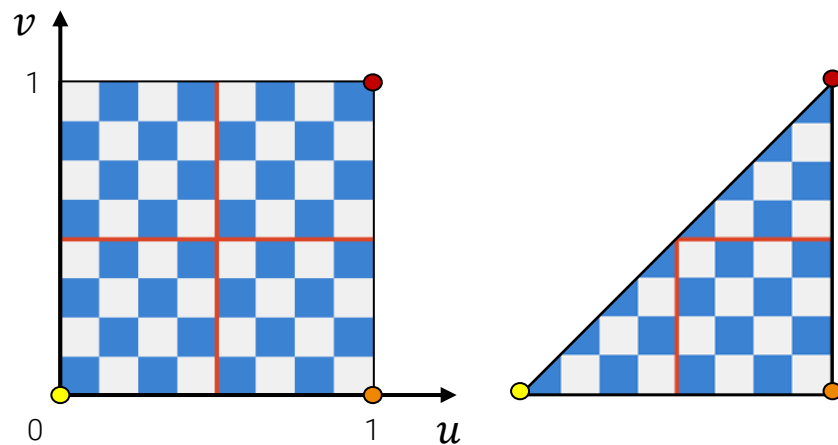
Texturzuordnung oder UV-Mapping

Ein 3D-Modell besteht zunächst nur aus Geometrie (Punkte im Raum). Jedem Vertex wird eine Position im Bild zugeordnet (**UV-Koordinaten**):

- $u \rightarrow$ horizontale Position im Bild (0 bis 1)
- $v \rightarrow$ vertikale Position im Bild (0 bis 1)

```
vt 0.0 0.0 → unten links  
vt 1.0 0.0 → unten rechts  
vt 1.0 1.0 → oben rechts  
vt 0.0 1.0 → oben links
```

```
f v/vt/vn  
f 1/1 2/2 3/3  
● ● ●
```



Baryzentrische Interpolation

Vollständiger OBJ-Parser in Processing

In der Vollversion werden im Ordner **data** neben der **obj**-Datei auch noch die **mtl**-Datei und eine **Texturdatei** eingelesen.

```
newmtl DemoMaterial
Kd 1.0 1.0 1.0
map_Kd checker.png
```

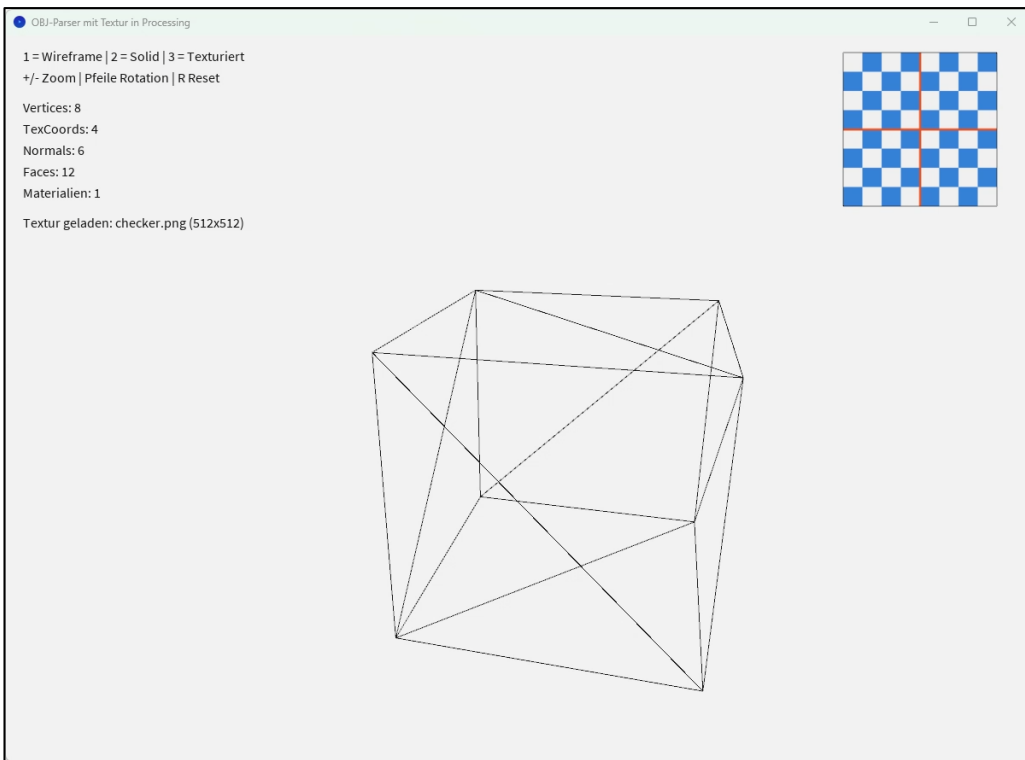
```
mtllib cube_textured.mtl
o TexturedCube
usemtl DemoMaterial
```

```
v -1 -1 1
v 1 -1 1
v 1 1 1
v -1 1 1
v -1 -1 -1
v 1 -1 -1
v 1 1 -1
v -1 1 -1
```

```
vt 0 0
vt 1 0
vt 1 1
vt 0 1
```

```
vn 0 0 1
vn 0 0 -1
vn -1 0 0
vn 1 0 0
vn 0 1 0
vn 0 -1 0
```

```
f 1/1/1 2/2/1 3/3/1 4/4/1
f 6/1/2 5/2/2 8/3/2 7/4/2
f 5/1/3 1/2/3 4/3/3 8/4/3
f 2/1/4 6/2/4 7/3/4 3/4/4
f 4/1/5 3/2/5 7/3/5 8/4/5
f 5/1/6 6/2/6 2/3/6 1/4/6
```





Schauen wir uns alternativ einen
OBJ-Parser in Java/LWJGL an

OBJ-Parser: Klassen FaceTriangle und Model

Wir können uns das in der Klasse **FaceTriangle** kurz anschauen, in der wir ein „Face“ speichern werden. Schauen wir uns ebenfalls kurz in die Klasse **Model** an:

```
import org.lwjgl.util.vector.Vector3f;

public class FaceTriangle {
    public Vector3f vertex    = new Vector3f();
    public Vector3f texCoords = new Vector3f();
    public Vector3f normal    = new Vector3f();

    public FaceTriangle(Vector3f vertex,
                        Vector3f texCoords,
                        Vector3f normal) {
        this.vertex    = vertex;
        this.texCoords = texCoords;
        this.normal    = normal;
    }
}
```

f 1/1/1 2/13/2 3/2/3

```
import java.util.ArrayList;
import java.util.List;
import org.lwjgl.util.vector.Vector2f;
import org.lwjgl.util.vector.Vector3f;

public class Model {
    public List<Vector3f> vertices = new ArrayList<Vector3f>();
    public List<Vector3f> normals  = new ArrayList<Vector3f>();
    public List<Vector2f> texCoords = new ArrayList<Vector2f>();
    public List<FaceTriangle> faces = new ArrayList<FaceTriangle>();

    public float size;
    public Model(){}
}
```

Die Klasse **POGL** hält eine Methode **loadModel** bereit, mit der Objekte im OBJ-Format eingelesen werden können.

OBJ-Parser: POGL-Methode loadModel

In der Klasse **POGL** können wir über die Methode **loadModel** eine OBJ-Datei einlesen und parsen:

```
public static Model loadModel(File file) throws FileNotFoundException, IOException {  
    ...  
    while ((line = reader.readLine()) != null) {  
        if (line.startsWith("v ")) {  
            lineElements = line.split(" ");  
            x = Float.valueOf(lineElements[1]);  
            y = Float.valueOf(lineElements[2]);  
            z = Float.valueOf(lineElements[3]);  
            model.vertices.add(new Vector3f(x, y, z));  
        } else if (line.startsWith("vn ")) {  
            lineElements = line.split(" ");  
            x = Float.valueOf(lineElements[1]);  
            y = Float.valueOf(lineElements[2]);  
            z = Float.valueOf(lineElements[3]);  
            model.normals.add(new Vector3f(x, y, z));  
        }  
        ...  
        model.faces.add(new FaceTriangle(vertexIndices, texCoordsIndices, normalIndices));  
    }  
    reader.close();  
    return model;  
}
```

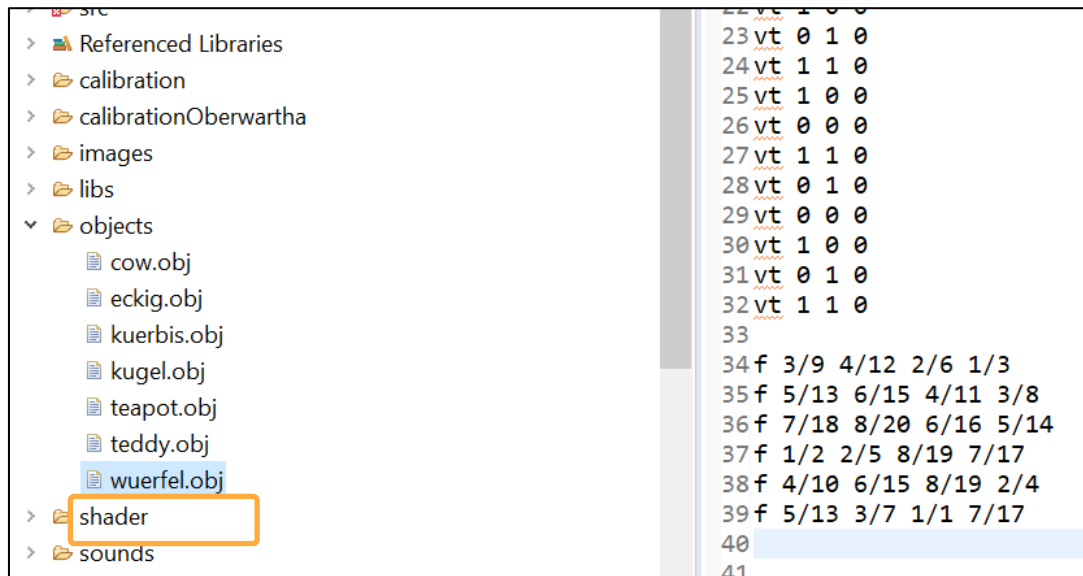
```
g Wuerfel  
v -0.5 -0.5 0.5  
v -0.5 0.5 0.5  
v 0.5 -0.5 0.5  
v 0.5 0.5 0.5  
v 0.5 -0.5 -0.5  
v 0.5 0.5 -0.5  
v -0.5 -0.5 -0.5  
v -0.5 0.5 -0.5  
  
...
```

```
f 3/9 4/12 2/6 1/3  
f 5/13 6/15 4/11 3/8  
f 7/18 8/20 6/16 5/14  
f 1/2 2/5 8/19 7/17  
f 4/10 6/15 8/19 2/4  
f 5/13 3/7 1/1 7/17
```



Beispielobjekte in der Programmsammlung

Ihr findet eine Reihe von Beispielobjekten im OBJ-Format in der Programmsammlung im Ordner **objects**.



Einfach die Objekte laden und die Funktionalität ausprobieren. Dazu einen Blick in die Klasse **ObjektLadenUndDrehen** werfen.

[1] Block-Berlitz, Marco: "Warum sich der Dino furchtbar erschreckte - Lehrbuch zu Beleuchtung und Rendering mit Java, LWJGL, OpenGL, OpenCV und GLSL", 3. Auflage, ISBN: 978-3-9819362-9-2, vividus Wissenschaftsverlag, 2021

Objekt laden und drehen

Jetzt wollen wir ein Objekt direkt in das Programm laden und drehen lassen:

```
public class ObjektLadenUndDrehen extends LWJGLBasisFenster {
    Model object = null;

    public ObjektLadenUndDrehen(String title, int width, int
        height, String fileName, float size) {
        super(title, width, height);
        JFrame f = new JFrame();
        f.setTitle(title);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        Canvas c = new Canvas();
        f.add(c);
        f.setBounds(100, 100, 600, 600);
        f.setLocationRelativeTo(null);
        f.setVisible(true);

        loadObject(fileName);
        object.size = size;
        initDisplay(c);
    }
}
```

```
public void renderLoop() {
    glPolygonMode(GL_FRONT_AND_BACK, GL_POINT);
    long start = System.nanoTime();
    while (!Display.isCloseRequested()) {
        double t = (System.nanoTime()-start)/1e9;
        POGL.clearBackgroundWithColor(0.95f, 0.95f, 0.95f, 1.0f);
        glLoadIdentity();
        glFrustum(-1, 1, -1, 1, 4, 10);
        glTranslated(0, 0, -5);
        glRotatef((float)t*50.0f, 0.0f, 1.0f, 0.0f);
        glScaled(1.0/object.size, 1.0/object.size, 1./object.size);
        glColor3d(0.35 + 0.30*Math.sin(t), 0.63 + 0.24*Math.sin(t),
            0.73 + 0.22*Math.sin(t));
        POGL.renderObject(object);
        Display.update();
    }
}
```

Programmcode: kapitel01/ObjektLadenUndDrehen.java

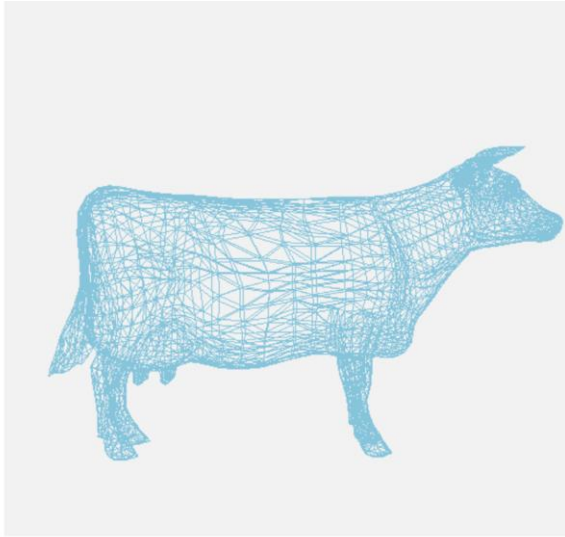
Darstellungsmodus wechseln

Um zwischen den drei Darstellungen (Punkte, Linien, Flächen) zu wechseln, schalten wir eine der Visualisierungen in der Methode **renderLoop** ein:

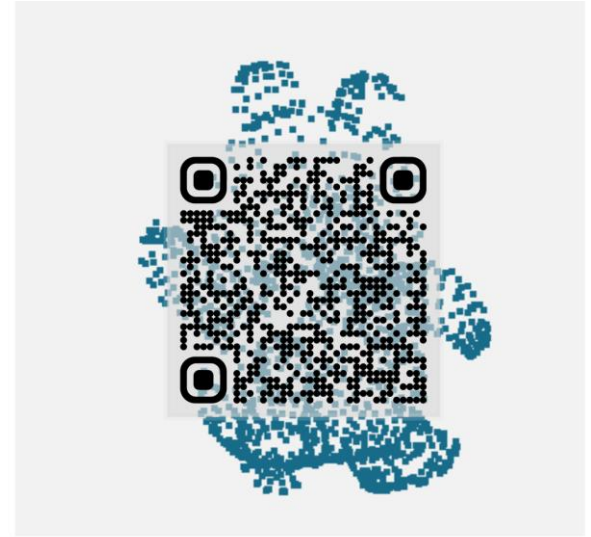
```
glPolygonMode(GL_FRONT_AND_BACK, GL_POINT);  
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);  
glPolygonMode(GL_FRONT_AND_BACK, GL_FILL);
```



teapot.obj



cow.obj



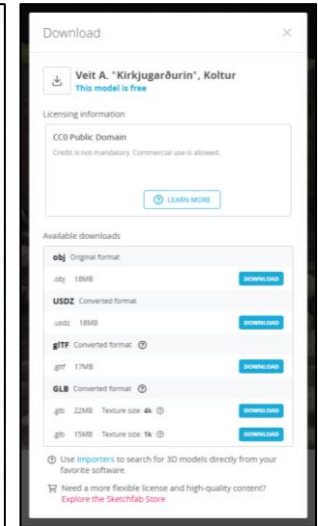
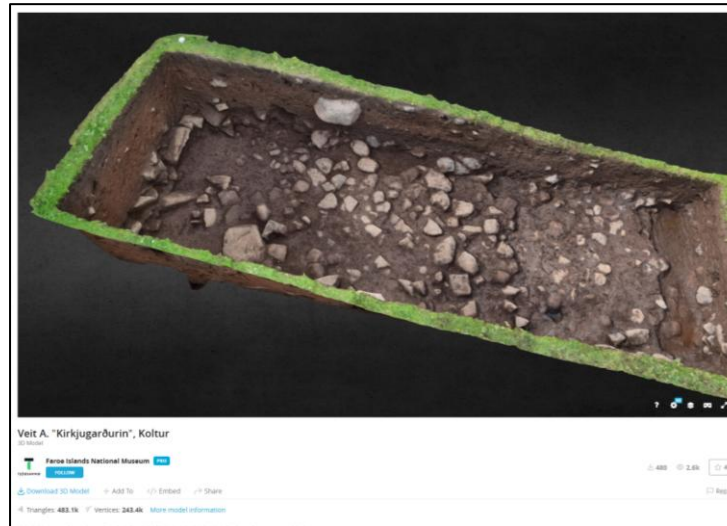
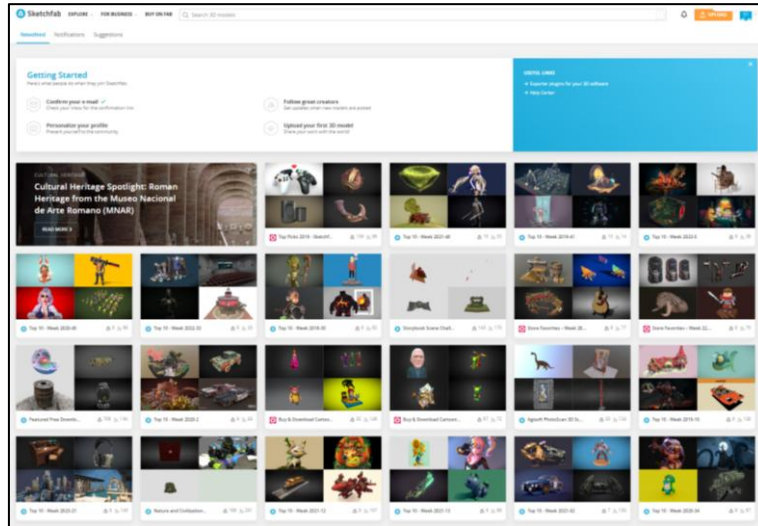
teddy.obj



Wo lassen sich jetzt interessante 3D-Modelle zum Anzeigen und Ausprobieren finden?

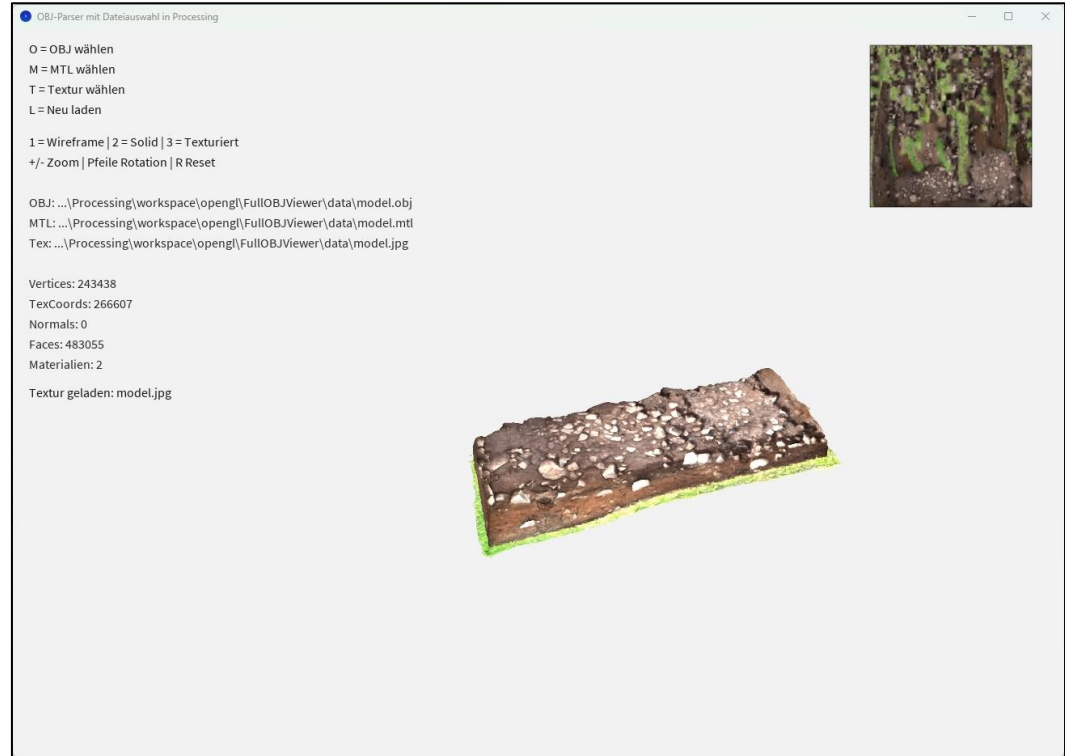
3D-Modelle bei SketchFab

Auf der Plattform [SketchFab](https://sketchfab.com) lassen sich verschiedene 3D-Modelle hochladen und veröffentlichen, aber auch herunterladen.



Vollständiger OBJ-Parser (mit Dateiauswahl) in Aktion

Hier sehen wir ein Beispiel für ein texturiertes 3D-Modell, das heruntergeladen und in mit dem OBJ-Parser eingelesen und angezeigt werden kann [1].



opengl/FullOBJViewer

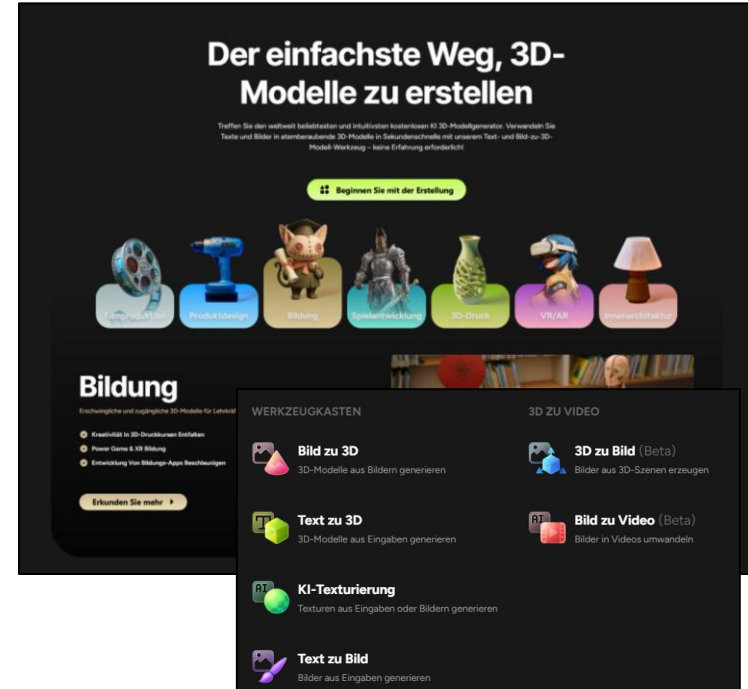
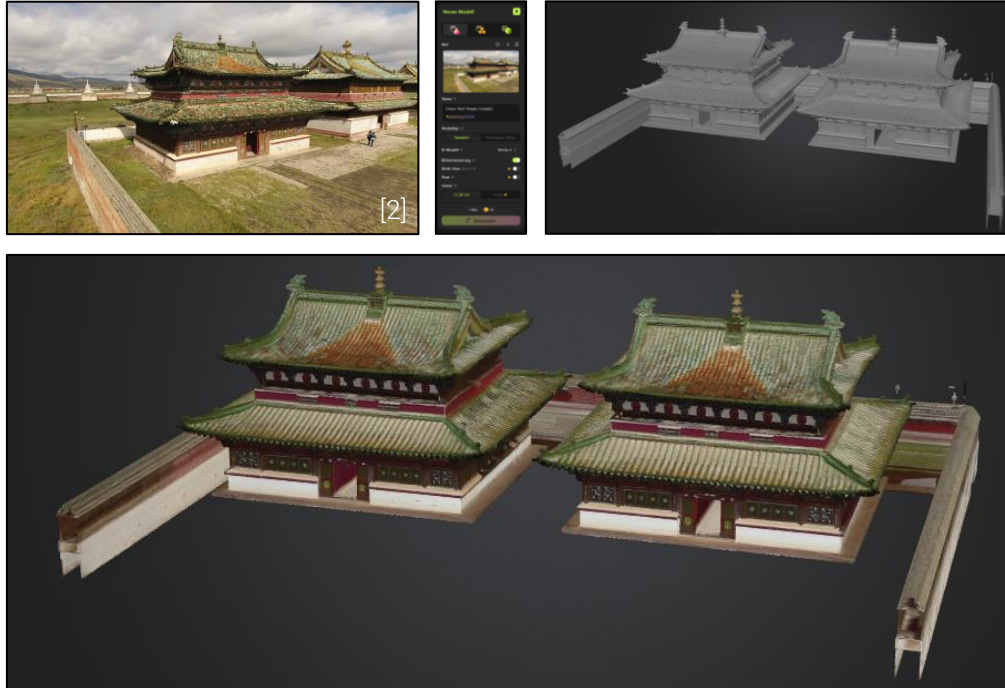
[1] Michelsen H.D.: „Veit A. "Kirkjugarðurin", Koltur", 2015, <https://skfb.ly/OyPV>



Modelle mit **GenAI** erzeugen

3D-Modelle mit Meshy.AI erzeugen

Mit **Meshy.AI** lassen sich 3D-Modelle aus Texten oder Fotos erzeugen und umgekehrt [1].



[1] <https://www.meshy.ai/de>

[2] Block-Berlitz M., Dücke B., Rohland H., Franken C., Suchowska P., Batbayar T., Erdenebat U.: "Area-Optimized, Rapid UAV-Borne Recording of Medieval Heritage in Central Asia", Journal of Field Archaeology, Taylor & Francis, 2021, pp. 1-15

3D-Modelle: Formate, Parsing und Visualisierung

Key Takeaways



- 3D-Modelle bestehen aus Geometrie, Topologie und Attributen
- Dreiecke sind die zentrale Repräsentation in der Computergrafik
- OBJ ist ein einfaches, textbasiertes Format für 3D-Daten
- Faces referenzieren getrennte Listen (Position, Normalen, Texturen)
- Komplexe Modelle benötigen Materialien und Texturen (MTL)
- Modelle können mit GenAI erzeugt werden

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Warum verwendet die GPU ausschließlich Dreiecke?
- (2) Was bedeutet die Notation **f v/vt/vn** konkret?
- (3) Warum sind OBJ-Dateien ineffizient für große Modelle?
- (4) Was passiert, wenn **vt** oder **vn** fehlen?
- (5) Warum müssen Quads trianguliert werden?
- (6) Unterschied zwischen OBJ und FBX?
- (7) Warum sind 1-basierte Indizes im OBJ problematisch?
- (8) Wie wird eine Textur einem Vertex zugeordnet?
- (9) Welche Fehler können beim Parsing auftreten?
- (10) Warum trennt OBJ Geometrie und Attribute?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Die bisherigen Visualisierungsbeispiele sollen um andere geometrische Formen erweitert werden. Wir hatten bspw. einen Würfel in OpenGL konstruiert und in der Funktion **renderWuerfel** in der Klasse POGL implementiert.

- a) Entwickeln Sie für die Klasse POGL die Funktion **renderKugel**, die eine Kugel im Bereich von -1 bis 1 darstellt und schreiben Sie eine Klasse **Kugel**, die diese Kugel in einer Rotation darstellt.
- b) Entwickeln Sie für die Klasse POGL die Funktion **renderStern(int n)**, die einen Stern mit n Zacken im Bereich von -1 bis 1 darstellt und schreiben Sie eine Klasse **Stern**, die diesen Stern in einer Rotation darstellt.
- c) Entwickeln Sie für die Klasse POGL die Funktion **renderPyramide**, die eine Pyramide im Bereich von -1 bis 1 darstellt und schreiben Sie eine Klasse **Pyramide**, die diese Pyramide in einer Rotation darstellt.

Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 2) [Java] Bei den vorhergehenden Aufgaben haben wir die Rotation eines Objekts über die Systemzeit realisiert. Erzeugen Sie weitere komplexere Bewegungsmuster.

- a) Realisieren Sie ein Bewegungsmuster ohne Rotation, das einer Acht entspricht.
- b) Realisieren Sie ein Bewegungsmuster mit beliebigen Rotationen um verschiedene Achsen, das einer Acht entspricht.
- c) Realisieren Sie ein weiteres komplexes Bewegungsmuster, das sich eignet, um erstellte 3D-Objekte aus verschiedenen Blickrichtungen darzustellen.

Aufgabe 3) [Java] Modellieren Sie in einem Tool Ihrer Wahl (Maya, Cinema 4D, 3DS Max, Blender, ...) ein interessantes, untexturiertes Objekt (bestehend aus Dreiecken), exportieren es im OBJ-Format und laden bzw. visualisieren es mit der Klasse **ObjektLadenUndDrehen**. Falls Sie Schwierigkeiten beim Einlesen des Formats haben, ergründen Sie die Problematik und passen den Code gegebenenfalls an.



Vielen Dank für die Aufmerksamkeit